

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Выполнил:
Тамразов Валерий
Группа J3211

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Привязать ранее разработанный в ЛР1 сайт к внешнему API средствами `fetch`. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах, рассмотренных в рамках тем "Имитация работы с API".

Выбранный вариант: 8. Платформа для автоматизации MLOps-процессов (CI/CD для моделей).

Ход работы

1. Настройка json-server с авторизацией

Для имитации бэкенда был установлен `json-server` совместно с `json-server-auth`, обеспечивающим авторизацию по JWT-токенам.

- Создан файл `package.json` с зависимостями `json-server` и `json-server-auth`.
- Создан файл `db.json` с моковыми данными: коллекции `users`, `pipelines`, `environments`, `artifacts`, `policies`, `roles`.
- Сервер запускается командой `npm start` на порту 3000.
- Защищённые эндпоинты доступны по префиксу `/600/`, требующему Bearer-токен в заголовке `Authorization`.

2. Реализация авторизации

Создан модуль `js/api.js` с функциями для работы с API: `getToken()`, `getUser()`, `isLoggedIn()`, `logout()`, `authHeaders()`, `checkAuth()`, `updateNavbar()`.

Страница входа login.html отправляет POST-запрос на /login с email и паролем. При успешном ответе токен и данные пользователя сохраняются в localStorage, после чего происходит редирект в личный кабинет.

Страница регистрации register.html отправляет POST-запрос на /register. Реализована клиентская валидация: проверка длины пароля (минимум 8 символов) и совпадения паролей. При ошибке API отображается сообщение.

- Если пользователь уже авторизован, страницы входа и регистрации автоматически редиректуют в личный кабинет.
- Защищённые страницы (профиль, пайплайны, среды) проверяют наличие токена через `checkAuth()` и перенаправляют на логин при его отсутствии.
- Навигационная панель динамически отображает email пользователя и кнопку "Выйти" для авторизованных, или кнопки "Войти"/"Регистрация" для гостей.

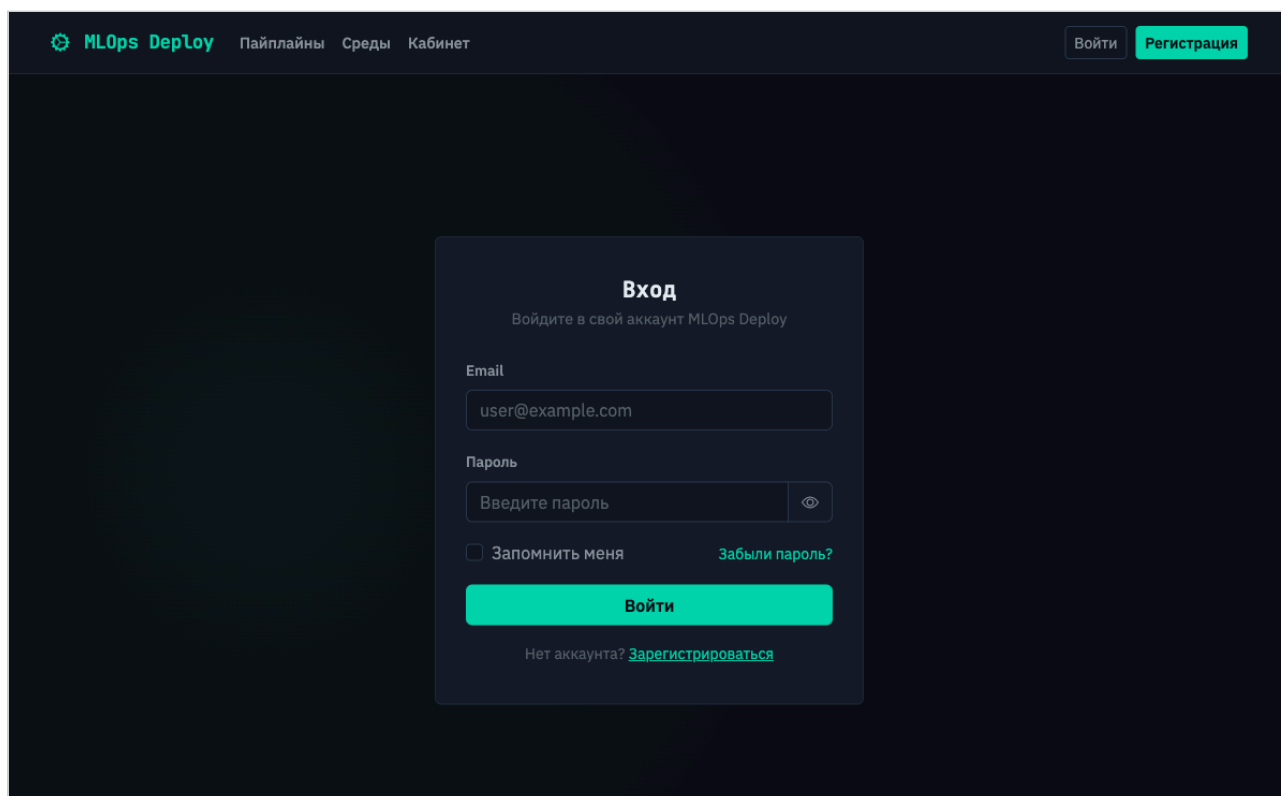


Рис. 1 Страница входа с отправкой данных через fetch

ML0ps Deploy Пайплайны Среды Кабинет Войти Регистрация

Регистрация

Создайте аккаунт для работы с платформой

Имя: Иван

Фамилия: Иванов

Email: user@example.com

Команда / Организация: Название команды

Роль: ML-инженер

Пароль: Минимум 8 символов

Повторите пароль: Повторите пароль

☐ Согласен с [условиями использования](#)

[Создать аккаунт](#)

Уже есть аккаунт? [Войти](#)

Рис. 2 Страница регистрации с валидацией и отправкой на /register

3. Загрузка пайплайнов из API

Страница pipelines.html загружает список пайплайнов из GET /600/pipelines с Bearer-токеном в заголовке. Карточки пайплайнов рендерятся динамически через JavaScript (DOM API).

- Каждая карточка отображает название, стратегию деплоя, владельца, среду, статус и прогресс-бар.
- Реализована клиентская фильтрация по загруженным данным: поиск по названию, фильтр по статусу (чекбоксы), среде и владельцу (select).
- При пустом результате фильтрации отображается сообщение "Пайплайны не найдены".
- Ссылки на пайплайны ведут на pipeline.html?id=N.

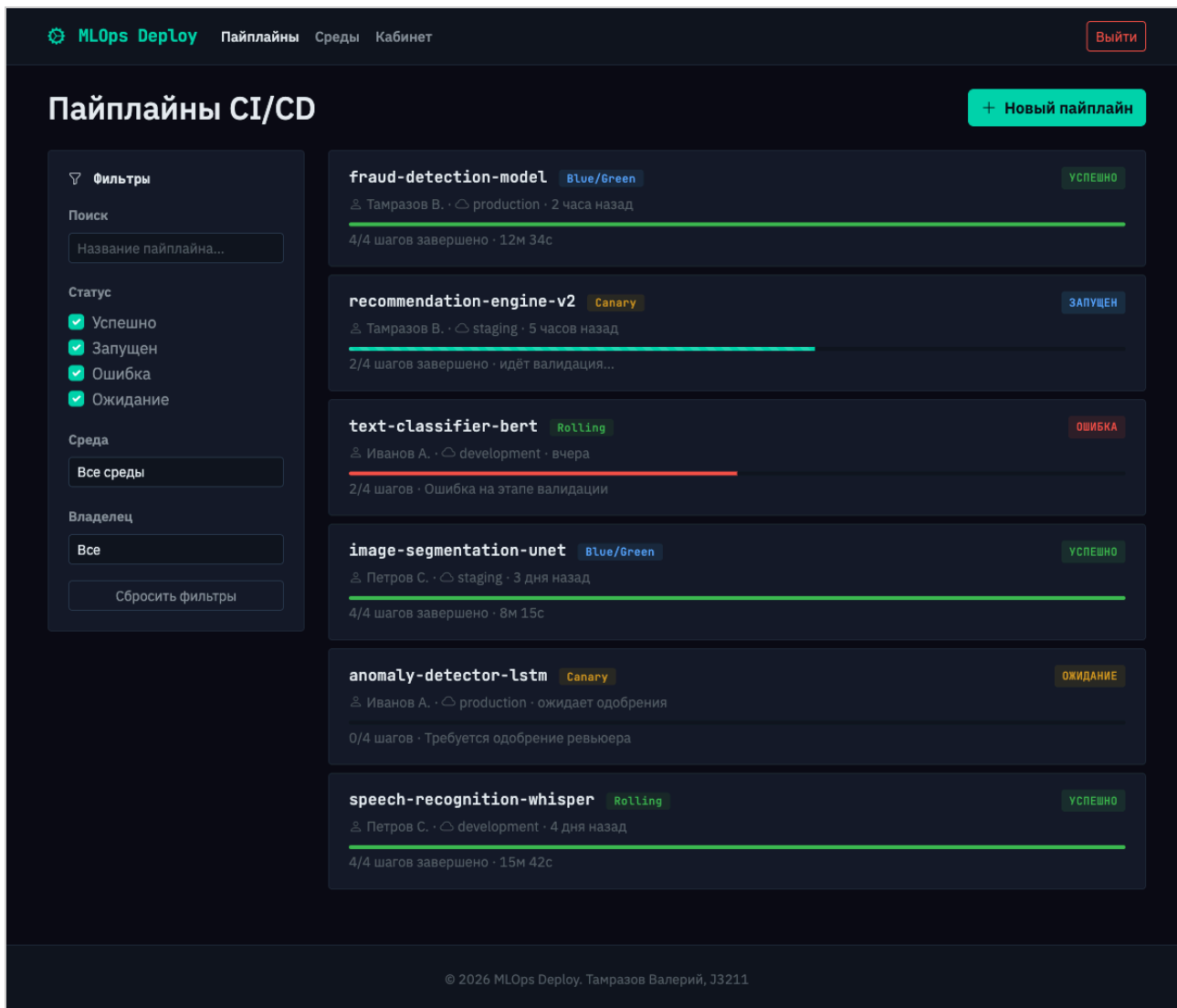


Рис. 3 Пайплайны, загруженные из API с фильтрацией

4. Загрузка данных профиля

Страница profile.html загружает данные из нескольких эндпоинтов:

- Имя, роль и email берутся из localStorage (сохранены при авторизации).
- Пайплайны пользователя: GET /600/pipelines?userId=N.
- Артефакты: GET /600/artifacts?userId=N.
- Политики: GET /600/policies?userId=N.
- Статистика (количество пайплайнов, артефактов, политик) вычисляется из полученных данных.

- Три вкладки ("Мои пайплайны", "Артефакты", "Политики") заполняются динамически.

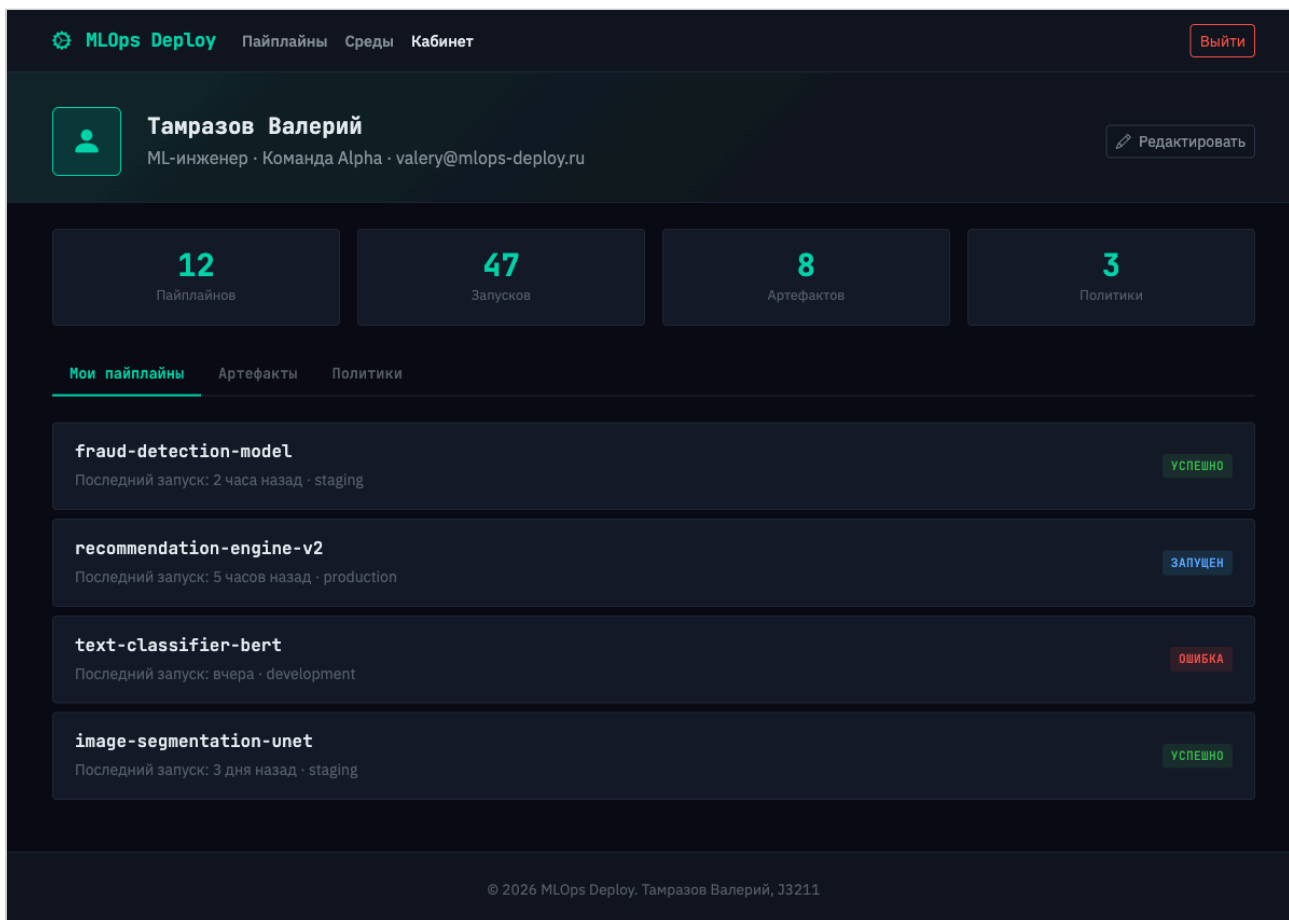


Рис. 4 Личный кабинет с данными из API

5. Страница пайплайна

Страница pipeline.html загружает данные конкретного пайплайна по параметру id из URL: GET /600/pipelines/:id.

- Название, статус, стратегия, среда, ветка и коммит заполняются из API.
- Шаги пайплайна (тестирование, валидация, упаковка, деплой) рендерятся динамически с учётом количества завершённых шагов.
- Артефакты загружаются из GET /600/artifacts?pipelineId=N.
- Боковая панель с информацией о пайплайне заполняется из ответа API.

6. Загрузка сред и ролей

Страница `environments.html` загружает данные из двух эндпоинтов:

- Среды: `GET /600/environments` — карточки с названием, стратегией деплоя, количеством моделей, RPM, uptime.
- Роли: `GET /600/roles` — таблица пользователей с ролями и уровнями доступа по средам.

MLops Deploy

Пайплайны

Среды

Кабинет

Выйти

Среды и роли

+ Новая среда

Production

Blue/Green

Активные модели: 5

Запросов/мин: 2,340

Uptime: 99.97%

Последний деплой: 2ч назад

Настроить

Staging

Canary

Активные модели: 8

Запросов/мин: 156

Uptime: 99.5%

Последний деплой: 5ч назад

Настроить

Development

Rolling

Активные модели: 12

Запросов/мин: 42

Uptime: 98.2%

Последний деплой: 1д назад

Настроить

Роли и разрешения

ПОЛЬЗОВАТЕЛЬ	РОЛЬ	DEVELOPMENT	STAGING	PRODUCTION	ДЕЙСТВИЯ
ВТ Тамразов Валерий valery@mlops-deploy.ru	Админ	✓ Полный	✓ Полный	✓ Полный	
АИ Иванов Алексей ivanov@mlops-deploy.ru	Разработчик	✓ Полный	✓ Деплой	👁 Просмотр	
СП Петров Сергей petrov@mlops-deploy.ru	Разработчик	✓ Полный	✓ Деплой	👁 Просмотр	
ЕК Козлова Елена kozlova@mlops-deploy.ru	Наблюдатель	👁 Просмотр	👁 Просмотр	👁 Просмотр	

Пригласить участника

© 2026 MLops Deploy. Тамразов Валерий, J3211

Рис. 5 Среды и роли, загруженные из API

Используемые технологии

- `json-server 0.17` + `json-server-auth 2.1` для мокового API с авторизацией

- Fetch API для HTTP-запросов к серверу
- localStorage для хранения JWT-токена и данных пользователя
- DOM API для динамического рендеринга контента (без innerHTML)
- Bootstrap 5.3 (CDN), Bootstrap Icons
- CSS3 с CSS-переменными, Google Fonts (JetBrains Mono, IBM Plex Sans)

Вывод

В ходе выполнения лабораторной работы сайт платформы MLOps Deploy был подключён к моковому API на базе json-server с авторизацией через json-server-auth. Реализованы регистрация и вход с сохранением JWT-токена, защита страниц от неавторизованного доступа, динамическая загрузка пайплайнов, артефактов, политик, сред и ролей из API с использованием Fetch API. Фильтрация пайплайнов работает по данным, полученным с сервера.